STOCKIQ · DOCUMENTAÇÃO DE MACHINE LEARNING

Machine Learning Mineração de Dados

Documentação do módulo analítico do StockIQ: o experimento de previsão de vendas por regressão (pré-processamento, seleção de atributos, modelos e resultados), sua integração ao sistema (simulação de vendas) e o analytics do backend.

Python 3 | pandas · NumPy | scikit-learn | Regressão | CRISP-DM

Gabriel da Silveira Pessoni · Lívia Portela Ferreira
FATEC Franca — Desenvolvimento de Software Multiplataforma · 6º semestre

Projeto Integrador VI
Computação em Nuvem II · Mineração de Dados

Sumário

01	**Visão geral**	Objetivo e papel no sistema
02	**Metodologia CRISP-DM**	Etapas e situação
03	**Base de dados**	Store Sales — Time Series Forecasting
04	**Pré-processamento**	Do dado bruto à tabela numérica
05	**Seleção de atributos e normalização**	Dez atributos por dois métodos
06	**Algoritmos de regressão**	Regressão Linear e Random Forest
07	**Resultados e discussão**	Desempenho no conjunto de teste
08	**Analytics integrado ao sistema**	O que a API já entrega
09	**Simulação de vendas no sistema**	Modelos do notebook servidos por API
10	**Execução e evolução**	Como rodar e próximos passos

01 Visão geral

O módulo de Mineração de Dados transforma histórico de vendas em previsões de demanda. O trabalho tem três frentes: (1) o **notebook** `Previsao_Vendas.ipynb`, que treina e compara modelos de regressão sobre a base pública *Store Sales — Time Series Forecasting*; (2) o **serviço de simulação** (`ml-service`), que serve esses modelos — sem alterar o notebook — ao backend e à tela *Simulação de Vendas*; e (3) o **analytics do backend** (`GET /api/analytics`), usado pela tela IA Analítica, com previsão por média móvel, matriz ABC×XYZ e sugestões de compra.

3 mi	97.680	41	0,92
REGISTROS NA BASE	LINHAS NO RECORTE	ATRIBUTOS	MELHOR R^2 (TESTE)

🧹 **Pré-processamento**
Limpeza, integração de tabelas, atributos de data e codificação.

🎯 **Seleção de atributos**
SelectKBest e RFE comparados, com padronização só onde necessário.

📈 **Dois algoritmos**
Regressão Linear (interpretável) e Random Forest (não linear).

📊 **Avaliação**
R^2, MAE e RMSE no conjunto de teste.

🧪 **Simulação no sistema**
Os modelos do notebook, servidos por API, alimentam a tela Simulação de Vendas.

📄 **Relatório ABNT**
`machine-learning/Relatorio_Previsao_Vendas.pdf` descreve o experimento em formato acadêmico.

02 Metodologia CRISP-DM

Etapa	O que foi feito	Situação
1 · Entendimento do negócio	Objetivo: prever vendas por dia, loja e família de produto para apoiar a reposição e reduzir rupturas	Concluída
2 · Entendimento dos dados	Seis arquivos CSV mapeados; qualidade verificada (duplicados, vazios, extremos, lojas ainda não abertas)	Concluída
3 · Preparação dos dados	Remoção do período anterior à abertura das lojas, recorte, integração (lojas e petróleo), datas, agrupamento de promoções e codificação	Concluída
4 · Modelagem	Regressão Linear e Random Forest; seleção de 10 atributos por SelectKBest e por RFE	Concluída
5 · Avaliação	R^2, MAE e RMSE no teste; comparação entre métodos de seleção; limitações registradas	Concluída (divisão aleatória)
6 · Implantação	Os modelos do notebook são servidos por `ml-service` e usados na tela Simulação de Vendas; a IA Analítica usa média móvel e ABC×XYZ calculados no backend	Simulação integrada

03 Base de dados

O experimento usa o dataset público *Store Sales — Time Series Forecasting* (competição do Kaggle, rede varejista Corporación Favorita, Equador). A estrutura — vendas diárias por loja e por família de produtos — é equivalente às saídas de estoque do StockIQ, o que permite validar o método antes de usar dados reais de operação.

Arquivo	Conteúdo	Registros	Uso
`train.csv`	Vendas por data, loja, família, valor vendido (`sales`) e itens em promoção (`onpromotion`)	3.000.888	Base principal
`stores.csv`	Cidade, estado, tipo (A–E) e agrupamento (`cluster`) de cada loja	54	Tipo e cluster
`oil.csv`	Preço diário do petróleo (`dcoilwtico`)	1.218 (43 vazios)	Preço do petróleo
`test.csv`	16 dias seguintes (16–31/08/2017), sem `sales`	28.512	Não usado
`holidays_events.csv`	Feriados e eventos	350	Não usado
`transactions.csv`	Transações por loja e dia	83.488	Não usado

O período vai de 01/01/2013 a 15/08/2017, com 54 lojas e 33 famílias. Como a base completa é grande, o notebook trabalha com um **recorte**: as lojas 1 a 5 (todas do tipo D; quatro em Quito e uma em Santo Domingo) e datas a partir de 01/01/2016 — **97.680 linhas**. Nesse recorte a venda média é de 532,8

unidades por registro, com mediana de 37 e máximo de 124.717; 10,5% dos registros têm venda zero. A distribuição é muito assimétrica: poucas famílias (mercearia, bebidas e hortifrúti) concentram o volume.

Coluna do dataset	Conceito no StockIQ	Uso
`date`	`Movement.createdAt`	Dia da semana, dia do mês e mês
`family`	`Category.name`	Variável categórica (codificada em colunas 0/1)
`sales`	`Movement.quantity` (saídas)	Variável prevista
`store_nbr` · `cluster` · `type`	Local de operação	Identificam a loja
`onpromotion`	Intensidade de promoção	Agrupada em 4 faixas

04 Pré-processamento



Figura 1 — Etapas do pré-processamento no notebook

#	Etapa	O que faz e por quê	Resultado
1	Remover o período anterior à abertura	Para cada loja, descarta as linhas anteriores à primeira venda: o zero ali é loja inexistente, não falta de demanda	−222.057 linhas na base completa (todas com venda zero)
2	Recorte	Lojas 1 a 5, a partir de 2016, para o treino rodar rápido	97.680 linhas
3	Duplicados e ausentes	Confere linhas repetidas e campos essenciais vazios	0 e 0
4	Integração	Junta tipo e cluster da loja; expande o petróleo para todos os dias do calendário (último valor conhecido nos fins de semana/feriados e até 31/08/2017)	sem valores vazios
5	Valores extremos	Pelo intervalo interquartil, [−484,31; 823,19]; os picos são grandes famílias, não erros, então **são mantidos** (diagnóstico)	14.016 fora do intervalo (~14%)
6	Agrupar `onpromotion`	216 valores distintos viram 4 faixas ordenadas: 0 · 1–3 · 4–16 · >16	57.484 · 17.301 · 13.257 · 9.638
7	Atributos de data	Dia da semana, dia do mês e mês (padrões como o fim de semana)	3 atributos
8	Codificação	Remove `id` e `date`; transforma família (33) e tipo da loja em colunas 0/1	41 atributos + alvo

> **OBSERVAÇÃO**
> As cinco lojas são do tipo D, então a coluna de tipo é constante e não contribui. A etapa 1 não altera o recorte das lojas 1 a 5 (operam desde 2013), mas mantém o fluxo correto se mais lojas forem incluídas.

05 Seleção de atributos e normalização

Os dados são divididos aleatoriamente em 70% treino (**68.376** linhas) e 30% teste (**29.304**), com semente fixa. A seleção usa **apenas o treino**, para que o teste não influencie a escolha. Dos 41 atributos, mantêm-se dez por dois métodos (GUYON; ELISSEEFF, 2003):

Método	Tipo	Como funciona	Atributos escolhidos
SelectKBest	Filtro	Pontua cada atributo isoladamente pelo teste F de regressão e retém os 10 melhores	`onpromotion` e 9 famílias (bebidas, livros, limpeza, laticínios, mercearia I, ferragens, eletrodomésticos, hortifrúti, material escolar)
RFE	*Wrapper*	Treina uma regressão linear padronizada, descarta o atributo menos importante e repete até restarem 10	`store_nbr`, `onpromotion`, `cluster` e 7 famílias (bebidas, limpeza, laticínios, mercearia I, utensílios II, hortifrúti, frutos do mar)

A **padronização** (média 0, desvio 1) é aplicada só à Regressão Linear, dentro de um *pipeline* que aprende os parâmetros no treino: atributos em escalas diferentes (número da loja × colunas 0/1) tornariam os coeficientes incomparáveis. A Random Forest não precisa disso, pois suas decisões dependem apenas da ordem dos valores.

06 Algoritmos de regressão

A variável prevista (`sales`) é numérica e contínua, o que caracteriza um problema de regressão. Foram escolhidos dois algoritmos de naturezas opostas, implementados com `scikit-learn`.

➗

Regressão Linear

A venda é uma soma ponderada dos atributos: $y = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p + \varepsilon$. Interpretável: cada coeficiente mostra o efeito de um desvio-padrão do atributo. Limitação: supõe relações lineares e aditivas.

🌲

Random Forest

Média de 50 árvores de decisão treinadas em amostras diferentes, com sorteio de atributos a cada divisão (BREIMAN, 2001). Captura não linearidades e interações sem normalização. Menos interpretável.

Métrica	Leitura
R^2	Fração da variação das vendas explicada pelo modelo (1 = perfeito)
MAE	Erro absoluto médio, em unidades vendidas
RMSE	Raiz do erro quadrático médio; penaliza mais os erros grandes

07 Resultados e discussão

Seleção	Algoritmo	R^2	MAE	RMSE
SelectKBest	Regressão Linear	0,6270	311,80	857,40
SelectKBest	Random Forest	0,6279	312,22	856,27
RFE	Regressão Linear	0,6337	345,62	849,67
RFE	**Random Forest**	**0,9156**	**172,82**	**407,87**

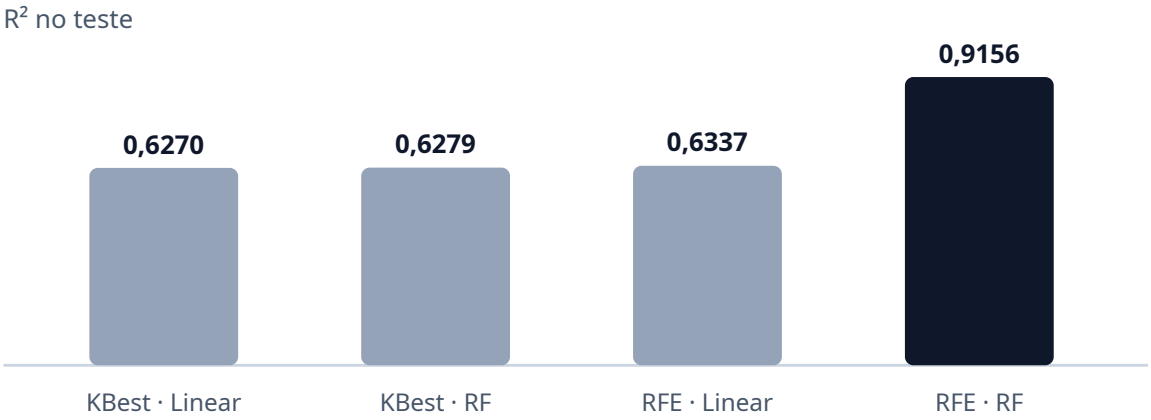


Figura 2 — R^2 por combinação de método de seleção e algoritmo

- **RFE + Random Forest foi claramente o melhor** (explica 91,6% da variação; erro médio de 172,8 unidades), contra R^2 perto de 0,63 nas demais combinações.
- Com o **SelectKBest** os dois algoritmos empataram: os dez atributos escolhidos são a promoção e famílias de produtos, que não distinguem uma loja da outra nem um dia do outro.
- O **RFE** incluiu o número da loja e o cluster; com eles a Random Forest pôde aprender o nível de venda de cada loja. A Regressão Linear quase não melhorou, o que sugere relação não linear.
- Treinando a Random Forest com **todos os 41 atributos** (verificação à parte, fora do notebook), o R^2 foi de 0,9518: o RFE perde cerca de 0,036 em troca de um modelo bem mais enxuto; o SelectKBest descartou informação decisiva.
- O RMSE bem maior que o MAE em todos os modelos indica erros concentrados em poucos registros de alto volume (picos de mercearia).

Equação da Regressão Linear (RFE, atributos padronizados)

Intercepto de **535,16**. Os maiores coeficientes (por desvio-padrão do atributo): mercearia I **+705,6**, bebidas **+573,4** e hortifrúti **+556,3**; `cluster` **−276,7** e `store_nbr` **−271,0** capturam o nível de cada loja.

> **LIMITAÇÕES**
>
> (1) A divisão treino/teste é **aleatória**, não cronológica: dias vizinhos aparecem nos dois conjuntos, o que torna o R^2 mais otimista do que seria prever dias realmente futuros. (2) Não há vendas passadas (*lags*), feriados nem transações como atributos. (3) O recorte tem só 5 lojas do mesmo tipo. (4) O `test.csv` não é usado nesta versão: o notebook avalia, mas não gera a previsão dos dias futuros.

08 Analytics integrado ao sistema

A tela **IA Analítica** consome `GET /api/analytics`, calculado pelo backend a partir das saídas reais (`movements`, tipo `exit`):

Recurso	Como é calculado
Previsão de demanda	Histórico de 12 semanas e projeção de 4 semanas por produto, com faixa de incerteza. A previsão vem de um serviço de ML **opcional** (`ML_SERVICE_URL`); sem ele — caso deste repositório, que não inclui o serviço Python — usa **média móvel**. `GET /api/analytics/model` informa a origem e as métricas do modelo, quando existe
Curva ABC	Produtos ordenados pelo valor movimentado; A até 80% acumulado, B até 95%, C acima
Classificação XYZ	Coeficiente de variação da demanda (CV = desvio ÷ média): X < 0,5 · Y até 1,0 · Z acima
Matriz ABC × XYZ	Cruza valor e regularidade, com giro anual e % do valor; orienta a política de estoque por célula
Sugestões de compra	Cobertura de 28 dias de demanda prevista + estoque de segurança, considerando o prazo de reposição (configurável); com urgência, motivo e custo estimado; exportável em CSV

	X · estável	Y · variável	Z · irregular
A · maior valor	Estoque de segurança baixo, reposição automática	Monitoramento frequente, margem de segurança	Atenção máxima: alto valor e demanda irregular
B · intermediário	Reposição periódica programada	Revisão periódica	Compra sob demanda com limite
C · menor valor	Lotes grandes, pouco controle	Controle simplificado	Avaliar descontinuação

Do experimento ao sistema

O notebook é a **técnica inicial de mineração de dados** do projeto: demonstra o método, os atributos que importam e o desempenho esperado. Seus modelos já estão ligados ao sistema pela simulação de vendas (próxima seção); o caminho seguinte é treinar com os dados reais de saída e validar com divisão cronológica.

09 Simulação de vendas no sistema

A integração foi feita **sem alterar o notebook nem a pasta** `machine-learning/`. O serviço `ml-service` (Python/FastAPI) lê as mesmas bases CSV e reproduz, na mesma ordem e com os mesmos parâmetros, o pré-processamento, a divisão 70/30 (`random_state=1`), a seleção de atributos e os quatro modelos. Um teste de equivalência confere que o recorte (97.680 / 68.376 / 29.304 linhas, 41 atributos), os atributos selecionados e o **R^2 dos quatro modelos são idênticos** aos do notebook.

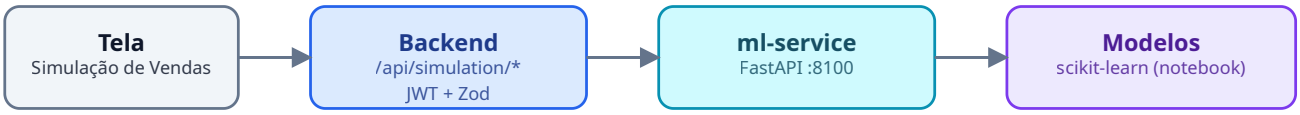


Figura 3 — Caminho da simulação, do navegador aos modelos

Rota (backend)	Função
`GET /api/simulation/status`	Fora do ar, treinando ou pronto (nunca falha)
`GET /api/simulation/model`	Modelos, métricas (R^2, MAE, RMSE), atributos selecionados, importâncias, lojas, famílias, faixas de promoção e petróleo
`POST /api/simulation/predict`	Venda diária prevista por cenário (loja, família, data, itens em promoção, petróleo) e por modelo — até 400 cenários

Modelo	Seleção	R^2	Usa a data?	Origem
Regressão Linear	SelectKBest	0,6270	não	Notebook
Random Forest	SelectKBest	0,6279	não	Notebook
Regressão Linear	RFE	0,6337	não	Notebook
Random Forest	RFE	0,9156	não	Notebook
Random Forest	41 atributos	0,9518	sim (dia da semana, mês, petróleo)	Complementar

- A tela permite alterar loja, família, itens em promoção, petróleo, data e horizonte e mostra: indicadores, previsão diária por modelo, o efeito de cada faixa de promoção, a comparação entre as 5 lojas e a tabela de desempenho.
- Os modelos do notebook (com seleção de atributos) **não usam a data**: a linha diária é constante. O modelo complementar — que **não** faz parte do notebook e é identificado como tal — varia por dia da semana (domingos vendem menos), mês e petróleo.
- Previsões negativas (possíveis na regressão linear) são cortadas em 0. Lojas válidas: 1 a 5; famílias: as 33 da base.

> **LIMITAÇÕES**
> Valem as do notebook: divisão treino/teste aleatória (R^2 otimista para dias futuros), sem vendas passadas, feriados ou transações, e só 5 lojas do tipo D. As 26 famílias que a seleção RFE não distingue recebem a mesma previsão nos modelos com seleção.

10 Execução e evolução

```
# notebook (experimento)
cd machine-learning
# bases em store-sales-time-series-forecasting/ (train.csv, stores.csv, oil.csv, ...)
pip install pandas numpy scikit-learn matplotlib jupyter
jupyter notebook Previsao_Vendas.ipynb

# serviço de simulação (usado pela tela Simulação de Vendas)
cd backend && npm run ml:service             # http://127.0.0.1:8100
cd ml-service && python3 -m unittest -v tests.test_equivalence   # confere R² idêntico ao notebook
```

Etapa	Ação	Ganho esperado
1	Dividir treino/teste **por data** (últimos dias para teste)	Métrica realista de previsão futura
2	Adicionar vendas passadas (*lags* ≥ 16 dias), médias móveis, feriados e transações	Maior poder preditivo para dias futuros
3	Usar todas as lojas e avaliar por loja/família	Generalização do modelo
4	Gerar a previsão do `test.csv` e comparar com a média móvel do backend	Evidência de ganho sobre o baseline
5	Treinar com os dados reais de saída (eventos de movimentação via S3/SQS) e levar o modelo à IA Analítica	Integração ponta a ponta com a previsão do sistema

REFERÊNCIAS

- BREIMAN, L. Random forests. *Machine Learning*, v. 45, n. 1, p. 5–32, 2001.
- GUYON, I.; ELISSEEFF, A. An introduction to variable and feature selection. *Journal of Machine Learning Research*, v. 3, p. 1157–1182, 2003.
- PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.
- COOK, A. et al. *Store Sales — Time Series Forecasting*. Kaggle, 2021.